

# ShopAssist

Voice-Enabled Shopping Support | Brief Project Report

27 September 2026 | External deployment pending

## 1. Objective and implementation

ShopAssist is an online-ready voice-enabled shopping-support chatbot. It accepts English speech or typed text, displays the recognized transcript, predicts one of 27 support intents using a trained BiLSTM, and generates controlled responses from fictional order records. The core flows are order tracking, cancellation, returns and refund tracking. Additional flows show invoices and change the address of an unshipped order. The application uses Python/Flask, PyTorch, SQLite, HTML, CSS and JavaScript.

## 2. Dataset and provenance

The source is Bitext Innovations' Customer Support 27K v11 dataset: 26,872 instructions labelled with 27 intents. It uses a synthetic/hybrid generation process, not transcripts collected from this application. The included projection retains row number, flags, text, category and intent; generic response text is excluded. Bitext-derived data is distributed with its CDLA-Sharing 1.0 license and attribution. Source Git blob: 404649b80bb5ec57463d7ce7e17bc48e63b01fbc.

## 3. Splitting and conversational supplement

After lowercasing, placeholder normalization and deduplication, 23,619 unique examples remain; 3,253 duplicate rows were removed and no conflicting normalized labels were found. A stratified seed-42 split yields 16,533 original training examples, 3,543 validation examples and 3,543 test examples. A first model scored 99.07% on the synthetic test set but failed short sanity-check questions. To address that observed weakness, 3,980 author-written/templated conversational training examples were added, giving 20,513 training records. Supplement matches to validation/test text were excluded. The same synthetic test split was revisited after development; it is not an independent final evaluation. Closely related synthetic templates can still occur across splits.

## 4. Speech recognition and application pipeline

Microphone audio is transcribed through the browser SpeechRecognition/webkitSpeechRecognition API with en-IN language, interim results and one utterance per recording. The transcript stays editable until the user sends it. The browser provider supplies the acoustic recognizer; no speech model is trained in this project. Recognition may use remote processing. The app handles unsupported browsers, denied permission, silence and network errors, with typed chat as fallback. Optional browser speech synthesis reads replies aloud. Pipeline: audio -> transcript -> token IDs -> BiLSTM -> intent -> dialogue state and SQLite lookup -> displayed/spoken response.

## 5. Deep-learning architecture

Text is Unicode-normalized and lowercased; numbers and dataset placeholders become the token entity. Word tokens are mapped to a training-only vocabulary of 995 entries, with padding and unknown tokens, and truncated to 40 words. The model uses Embedding(64), a single bidirectional LSTM with 64 hidden units in each direction, concatenated final hidden states (128 dimensions), dropout 0.3, Dense(64) with ReLU, dropout 0.2 and Dense(27). Softmax gives intent scores. There are 140,251 trainable parameters. The BiLSTM learns sequence features; the separate dialogue manager does not replace the learned classifier with keyword routing.

## 6. Training and prediction policy

Training uses multiclass cross-entropy, AdamW (learning rate 0.002, weight decay 0.0001), batch size 128, gradient clipping at 1.0 and seed 42. Training ran 10 epochs, with the best validation macro-F1 checkpoint from epoch 7. PyTorch 2.14.0+cpu ran on CPU with two threads; the measured training duration was 46.56 seconds in the build environment. Validation selects the lowest tested score threshold with at least 98% accuracy among accepted validation predictions, subject to a fixed 0.15 margin. The selected threshold is 0.55; deployment also requires at least 50% vocabulary coverage. These are heuristic rejection rules, not calibrated confidence or a dependable out-of-domain detector.

## 7. Conversation state and database behavior

A signed, opaque browser session identifies isolated SQLite state containing five fictional orders. Missing order IDs, return reasons and new addresses are collected in follow-up turns. Exact confirmations and control replies are handled deterministically. The classifier handles new support requests. Cancellation requires processing status; return requests require delivery within 14 days. Each change is proposed before confirmation. Shipped orders cannot be cancelled. Refund status is read from records, not invented by a language model. Request IDs make retries idempotent. Reset affects only the current session. No real purchases, payments, emails, human-agent transfers or return pickups occur.

## Measured results

| Metric                                          | Measured result                |
|-------------------------------------------------|--------------------------------|
| Synthetic held-out text accuracy                | 99.13% (n=3,543)               |
| Synthetic held-out macro-F1                     | 0.9898                         |
| Accepted synthetic test coverage                | 99.63%                         |
| Accepted synthetic test accuracy                | 99.35%                         |
| New authored challenge accuracy                 | 60.0% (n=40)                   |
| Unrelated inputs rejected                       | 12/20                          |
| Median CPU inference on challenge               | 1.08 ms (environment-specific) |
| Functional API/state tests                      | 17/17 passed                   |
| Speech WER / live browser / external deployment | Not yet verified               |

## 8. Testing and limitations

All 17 automated functional tests passed with the actual trained classifier. They cover tracking, positive/negative confirmations, cancellation eligibility, returns, expired return windows, address changes, invoices, unknown orders, intent switching, session isolation, retries and invalid input. JavaScript syntax checks passed. A separate author-written challenge of 40 new questions across 10 intents scored 60.0% accuracy. This challenge was not used for model fitting or threshold selection, but it is still small and not independently sourced. The model accepted 37 of 40 questions; only 23 accepted predictions were correct. It rejected 12 of 20 unrelated probes. Thus the synthetic held-out score substantially overestimates performance on unfamiliar phrasing. Short paraphrases, missing vocabulary, mixed intents and implicit references remain weaknesses.

## 9. Deployment and remaining validation

Dockerfile, Gunicorn entry point and render.yaml are included for an external public Render web service. The frontend and API share one origin. The service binds to the supplied PORT, exposes /health and runs one worker with four threads. The model checkpoint ships with the app, so deployment does not train or download a model. SECRET\_KEY should be a host-managed secret and COOKIE\_SECURE=1 on HTTPS. Free-host restarts can reset the disposable SQLite database and cold starts may delay the first request. No successful external deployment or live URL is claimed at report creation; deployment remains pending. Browser UI and microphone accuracy are also unverified because the separate browser-preview environment could not run the Python application.

## 10. Speech evaluation and future work

Before submission, test the public HTTPS link on a real device. Record at least 20 unedited spoken test transcripts and calculate word error rate: (substitutions + deletions + insertions) / reference words. Also measure intent accuracy from those transcripts and end-to-end latency. None of these speech measurements has been completed; CPU classifier latency is not a substitute. Future improvements include independently collected support questions, stronger semantic representations, explicit out-of-domain examples, calibrated rejection and a broader untouched external test set. A production deployment would additionally need real authentication, authorization and durable storage.

## References

Bitext dataset: <https://huggingface.co/datasets/bitext/Bitext-customer-support-llm-chatbot-training-dataset>

Bitext source: <https://github.com/bitext/customer-support-llm-chatbot-training-dataset>

SpeechRecognition: <https://developer.mozilla.org/en-US/docs/Web/API/SpeechRecognition>

Render Docker: <https://render.com/docs/docker>

Render free-instance behavior: <https://render.com/docs/free>